

命中率指标掩盖的慢介质全量重读：MoE 推理平台的实证分析与排查判据

作者姓名

作者单位，省份 城市 邮编；作者职称、研究方向占位

收稿日期：2026-××-×× 基金项目：××××（编号×××××××）

第一作者：×××，××（职称），主要研究方向为×××；邮箱：xxx@xxx.xxx（通讯作者）

中图分类号：TP3 文献标志码：A

摘要：混合专家（Mixture-of-Experts, MoE）大模型的推理平台中，一个常被忽视的带宽问题在于：当被激活的专家权重无法常驻最快介质层时，即使引擎报告的缓存命中率为 100%，每个 token 仍必须从慢介质全量重读全部专家权重。本文以 2.8T 参数开源模型在 k3 x86 主机上的冷启动推理为对象，通过字节级台账实测了这一现象：每 token 专家段从慢盘读取 25.83 GB，耗时 303.58 s，占端到端耗时的 94%，而平台自带“命中率”白板指标并未反映这一事实。本文将问题归因于“复用未落在数据能到达的最快介质层”，给出形式化判据：若复用确实发生在最快介质层，则每个更慢介质层的最小读取次数为 1，且通过缓存调度（如将权重迁入较快介质）可达。在同一平台上的对照实验中，将专家权重全量迁入 L2 介质后，专家段耗时降低约 37%，方向与判据一致。本文实测数字仅对本实验成立，判据与排查次序可迁移到同类受内存约束的推理平台。

关键词：混合专家模型；大模型推理；缓存命中率；存储层次；带宽墙；工程实测

1 引言

1.1 背景与问题

大语言模型规模持续扩张，MoE 凭借稀疏激活成为主流扩展范式[1-6]。以 2.8T 参数开源模型为例[1]：共 93 层，896 路由专家，每 token 激活 16 个专家，激活参数约 104B。单看稀疏红利，每 token 只需读取全部专家权重的约 1.8%——这是 MoE 推理“省带宽”承诺的由来。

但该承诺有一个常被忽略的隐藏前提：被激活的权重必须常驻在或能快速到达计算侧。在 GPU 服务器上，trunk 与热专家常驻显存，前提成立；而在仅有系统内存、需从机械盘流式读权重的主机或边缘设备上，前提不成立。此时稀疏只决定“必须搬动多少字节”，没有决定“这些字节从哪一层介质搬”。

这种平台上，管理层与工具层用了同一类指标（缓存命中率、缓存命中计数）汇报“缓存状态”，但这些白板指标与底层字节搬迁事实是两个观测面，可能互相遮蔽。本文用一个真机案例说明这种遮蔽可严重到什么程度，并给出一条可迁移的排查判据。

1.2 实测发现：命中率 100%，字节却每 token 全量重读

在一次 k3 x86 主机的冷启动推理测试中（2026-08），引擎自报缓存命中率 100%。同一时段的内建字节级 READ 计数（台账，每行可回查）却显示：

每 token 专家段从慢盘 sde 读取 25.83 GB (=92 层 $\times$ 16 专家 $\times$ 17.55 MB/专家，与理论载入逐字节吻合)；该 25.83 GB 以约 85 MB/s 计耗时 303.58 s，占端到端 324.36 s 的 94%[13]。

也就是说：报告口径说“全部命中”，字节口径说“每 token 全量重读一遍”。引擎并没有做错什么——它的资源配置决定了被激活专家不常驻更快介质，因此每次都必须回慢盘取数；问题在于“命中率 100%”这个被普遍用作健康度的指标，完全看不到这一点。

1.3 本文工作

- ① 提供可逐字节回查的字节流台账，揭示“命中率白板指标与慢介质全量重读并存”的现实；
- ② 把现象归因提炼为一条可排查的判据（复用必须落在数据能到达的最快介质层），并给出形式化表述与证明；
- ③ 用对照实验（专家迁入 L2 介质，专家段耗时 - 37%）验证判据的方向有效性，给出可迁移的排查次序。

2 相关工作

2.1 MoE 稀疏激活与它的带宽前提

MoE 思想可追溯至 Shazeer 等的稀疏门控网络[2]，GShard[3] 提出 top-K 路由与分片，Switch Transformer[4] 将激活专家降至单个，Mixtral of Experts[5] 验证开源 $8 \times 7B$ 级 MoE 的实用性，DeepSeekMoE[6] 提出细粒度专家切分与共享专家隔离，本文实验对象[1]在此基础上将路由专家扩展至 896。上述工作的稀疏红利都以“激活权重可被快速访问”为前提；本文不讨论模型侧稀疏性优化，聚焦“既定的激活集合如何穿越存储层次”这一实现侧问题。

2.2 存储层次与带宽建模

Roofline 模型[7]给出了算力/带宽瓶颈的经典判断框架：在带宽受限区（decode 阶段约 1 FLOP/byte），复用是唯一杠杆。但 Roofline 回答的是“复用值不值得做”，未回答“复用应落在哪一层介质”。Eyeriss[8] 在加速器层面实证了“慢层读一次、片上复用数百次”的存在性（AlexNet 上 DRAM 访问约 0.0029 次/MAC）。这与本文方向一致；本文补充的是工程观测面：如何发现平台的“复用”其实没有落在应落的那一层。

2.3 LLM 推理的缓存、预取与调度

vLLM/PagedAttention[9] 用分页缓存消除 KV 碎片，可视为“让复用尽量落在快层”的实现；FlightLLM[10] 在 FPGA 上把 decode 权重复用约束在片上；CXL-SpecKV[11] 用 CXL 内存池、预测预取与冷热分层缓解 KV 缓存带宽墙，其活集超容量时的预取/驱逐正是本文判据“不在场”情形的处置；FastKV[12] 以“只保留必要数据于快层”为原则解耦压缩与算力。这些工作都隐含“慢层应只被读一次、复用应留在快层”的工程直觉，但未见把它变成一条可回查的判据；本文补上这形式化一步。

3 排查判据：慢介质读一次

本节是一条可回查的判据，作为排查工具使用。若某平台实测的慢层读取次数明显大于其下界，说明判据的“在场条件”未被满足——先查复用落在哪一层，而不是先怀疑算法是否需要免读。

3.1 定义

设存储层次由 N 层介质构成，按单次字节访问代价单调上升排序： M_1 （最慢，代价最高） $\rightarrow M_2 \rightarrow \dots \rightarrow M_N$ （最快，代价最低）。对数据项 d ，被访问（复用）总次数 $R \geq 1$ ；记 R_i 为 d 从介质 M_i 被读取的次数（ $i=1, \dots, k$, M_1 最慢）。

参与路径：判据只对 d 实际穿越的介质下结论。某介质物理上更快但 d 从不进入，则不参与 d 的路径，不计入求和范围。

3.2 判据（命题）

设数据项 d 被复用 $R \geq 1$ 次，且复用的实际发生位置是 d 能到达的最快介质 M_k 。则对任意 $i < k$ ，恒有

$$R_i \geq 1,$$

且该下界同时可达：存在读取调度使 $R_i=1$ 对所有 $i < k$ 同时成立，其余 $R-1$ 次全部发生在 M_k 。

推论：复用若发生在最快层，各慢层读取次数总和的最小值即“各慢层恰读一次”；使该最小值达成的调度同时是慢访问代价最小的调度。

3.3 证明

第1步（信息守恒下界，无条件）：若 d 从未被从 M_i （ $i < k$ ）读入，则 d 不可能到达 M_{i+1} 及以上层次，更不可能到达 M_k 被复用——数据只能逐层向上搬移。因此每层 M_i （ $i < k$ ）至少被读一次： $R_i \geq 1$ 。此下界不依赖任何容量假设与工作量。□

第2步（代价单调）：介质按访问代价单调排序，层号越大代价越低。任何越过下界1的“多余”慢读都可被替换为一次更快介质的读取而不增加总代价（慢读严格贵于快读）。故达到下界的调度使慢访问总代价最小。□

第3步（可达性）：复用发生在 M_k 意味着 d 在 M_k 有活集位置， $R-1$ 次复用可全部在 M_k 内完成； d 到达 M_k 的路径即逐层各读一次（ M_1 一次 $\rightarrow M_2$ 一次 $\rightarrow \dots \rightarrow M_k$ 一次），该调度合法且同时满足所有下界。□

3.4 判据的使用与边界

用法：先核对“复用事实是否真的发生在最快层”（在场条件）；再数“慢层实测读取次数”。若实测明显大于1，几乎总是因在场条件被违背（典型如“命中不落内存、每次仍全量搬迁”）。此时应去查资源配置，而不是先质疑算法。

边界：容量不构成判据条件——若活集超出 M_k 容量，复用无法全部落在 M_k ，这是“不在场”情形，判据不违反、不适用；物理更快但不参与路径的介质不计入；判据回答“下界是多少、达到没有”，不回答“如何达到”（那交给调度工作[9-12]）。

4 真机实测与对照实验

4.1 平台与数据来源

实验对象：2.8T 参数开源模型权重[1]。作者对权重做 safetensors 全量字节扫描，专家语义为 MXFP4 格点、实际落盘约 2.12 bit/weight（含 scale）[13]，单专家 17.55 MB。引擎：k3 x86 真机推理实现[14]；存储层次：慢盘 sde（源盘，近满、实测约 84~85 MB/s）→ L2 介质 sdd7 → 系统内存 → 计算。运行时统计内建字节级 READ 计数；出处为台账文件，标注行号可逐字节回查[13]。

4.2 发现一：命中率 100% 与“每 token 全量重读”并存

表 1 给出关键台账数字。核心结论是两套口径的分裂：白板口径（引擎自报）= 缓存命中 100%；字节口径（台账实测）= 每 token 从慢盘全量重读 25.83 GB、占端到端 94%。且实测 READ 量与理论载入逐字节吻合（ $92 \times 16 \times 17.55 \text{ MB}$ ），说明不是“额外读取”，而是“该读的一次都不少、还从最慢的介质读”。

表 1 专家段基准台账（冷启动 2026-08）

指标	数值	出处
每 token 专家理论载入	$92 \times 16 \times 17.55 \text{ MB} = 25.83 \text{ GB}$	台账 :246
实测专家段 READ/token	25.83 GB	:325, :327
专家段耗时	303.58 s/token	:327
端到端耗时	324.36 s/token	:325 (total)
端到端占比	$303.58/324.36 = 94\%$	:327
等效速率（精确商）	$25.83 \text{ GB}/303.58 \text{ s} = 85 \text{ MB/s}$	:327
介质刻画速率（碰壁整值）	84 MB/s（D-state 根因刻画）	:224

说明：表中 84 MB/s 与 85 MB/s 两值各有出身——84 MB/s 是介质性能刻画的整值（慢盘近满、巨型 O_DIRECT pread 的实测形态），85 MB/s 是 25.83 GB 与 303.58 s 的精确商。本文据实并列，不引入第三个“速率”声称。

4.3 为什么白板指标“看不见”这一事实

对照第 3 章判据：该平台的问题是“在场条件”被违背——被激活专家的复用没有落在它能到达的最快介质（内存）层，而是每次落回慢盘。引擎自报的“命中率”建立在白板计数口径上，这一口径与字节搬迁是两个观测面，前者无法反映后者。由此得到一条经验：对受内存约束的推理平台，“缓存命中率”不应单独作为健康度指标，需与字节级 READ 计数对照使用。

4.4 对照实验：让复用靠近快层

判据预演：慢介质读一次的前提是复用落在更快层。2026-08 复查将专家全量 distinct 集（约 10,010 个专家 $\times$ 17.55 MB $\approx$ 176 GB）驻留到 L2 介质 sdd7（实测约 419 MB/s，较慢盘快约 5 倍）：专家段耗时从 303 s 降至 191.6 s（约 -37%），端到端从 326.5 s 降至 262.8 s。

该读数两个方向性含义：其一，方向与判据一致——把慢盘全量重读移到更快介质，显著压缩代价；收益未达判据给的上界（trace 推演 25.8 GB→2.58 GB/token，约 -90%），因为复用仍未

真正落进内存层（每 token 仍从 L2 介质 pread 25.83 GB, 1786 MB/s 下亦需约 14.5 s），白板命中同样不能免除字节搬运。其二，判据得到印证——瓶颈不由“算法能否免读”决定，而由“复用发生在哪一层”决定。

4.5 方法局限

第 3 章判据属逻辑层，其真值不依赖本节任何数字；本节台账属真机层，账实一致、可回查，但只对本实验（该模型、该介质、该负载、该冷启动时序）成立。台账中如实记录挂死/FAIL 条目（如 D-state 永久挂起），未选择性剔除负样本。

5 讨论与应用

5.1 排查次序：先问“复用落在哪一层”

- ① 算法能否免读：该数据项是否必须被读取（如专家是否真是路由集合的必要输入）；
- ② 能否缓存复用：读取结果能否在快层驻留并被复用（即判据在场条件）；
- ③ 落盘位置：若必须慢读，权重是否至少置于参与路径中较快的介质；
- ④ 落盘前压缩：能否先压缩再落盘，减少届时必须搬动的字节。

本案例中引擎在第 ② 条失守（命中不落内存、每次仍全量搬迁），纵使第 ①、④ 条成立，带宽墙仍占端到端 94%。

5.2 与调度/预取工作的衔接

本文判据不替代调度器，而是为既有调度工作[9-12]提供可回查的落点：当方案声称“缓存命中”或“预取成功”时，应核实字节层是否真的避免了慢层重读——命中率指标与非易失介质上的字节搬迁指标可能互相遮蔽（见 4.4）。这正是本文想提醒平台开发者与评测者的一条操作步骤。

5.3 局限性

存储模型假设单向逐层搬移，未覆盖旁路、直接 DMA 到计算侧等非逐层实现路径——“逐层”是可达性的充分构造，非必要路径。

真机数字来自单一模型、单次冷启动实测，不声明跨工作量恒真；判据可达性由第 3 步构造保证，与读者是否阅读本文无关。

6 结论

本文通过逐字节可回查的真机台账揭示了一个在受内存约束的 MoE 推理平台上真实存在的现象：引擎自报缓存命中率 100% 的同时，每 token 仍从慢盘全量重读 25.83 GB 专家权重，占端到端耗时的 94%。“命中率”白板与字节搬运是两个观测面，前者可能完全遮蔽后者。本文从该现象抽象出一条可迁移的排查判据（复用必须落在数据能到达的最快介质层，否则慢层读取次数必然大于其下界¹⁾），并用专家迁入 L2 介质后的 -37% 对照实验验证了判据的方向有效性。对同类平台的开发者与评测者，本文建议把“复用发生在哪一层介质”作为排查带宽墙的第一步，并与“缓存命中率”指标并行观测。

参考文献

- [1] Moonshot AI. Kimi K3: Open Frontier Intelligence. arXiv:2607.24653, 2026.
- [2] Shazeer N, Mirhoseini A, et al. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. ICLR 2017.
- [3] Lepikhin D, Lee H, et al. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. NeurIPS 2020.
- [4] Fedus W, Zoph B, Shazeer N. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. JMLR 23(120):1-39, 2022.
- [5] Jiang A Q, Sablayrolles A, et al. Mixtral of Experts. arXiv:2401.04088, 2024.
- [6] Dai D, Deng C, et al. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. ACL 2024:1280-1297.
- [7] Williams S, Waterman A, Patterson D. Roofline: An Insightful Visual Performance Model for Multicore Architectures. Communications of the ACM, 52(4):65-76, 2009.
- [8] Chen Y H, Emer J, Sze V. Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks. ISCA 2016:367-379.
- [9] Kwon W, Li Z, et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023:611-626.
- [10] Zeng S, Liu J, et al. FlightLLM: Efficient Large Language Model Inference with a Complete Mapping Flow on FPGAs. FPGA 2024.
- [11] Liu D, Yu Y. CXL-SpecKV: A Disaggregated FPGA Speculative KV-Cache for Datacenter LLM Serving. FPGA 2026:56-66.
- [12] Jo D, Song J, Kim Y, Kim J-J. FastKV: Decoupling of Context Reduction and KV Cache Compression for Prefill-Decoding Acceleration. Findings of ACL 2026.
- [13] 作者. kimi-k3-in-c: Kimi K3 推理引擎实现与本文字节流台账分析代码.
<https://github.com/openchat-ai/kimi-k3-in-c>, 2026.
- [14] 作者自建真机字节流台账 (k3 x86 冷启动, 2026-08), 项目内文件: notes/bytflow-matrix.md.